

LLM & AI Agent Foundations

Core concepts to know before building AI agents

1. How LLMs Work (Black-Box Level)

You don't need to know transformer math to build agents well — you need working intuition for how a model consumes input and produces output.

Tokens & Tokenization

LLMs don't read words — they read **tokens**, which are chunks of text (often parts of words). A rough rule of thumb: 1 token $\approx$ 4 characters of English text, or about 0.75 words.

- Both your prompt and the model's reply are measured in tokens.
- You are billed (via API) per token, both input and output.
- Uncommon words, code, and non-English text often use more tokens per word.

Context Window

The **context window** is the maximum number of tokens (input + output combined) a model can handle in one call. Everything the model "knows" during a conversation — system prompt, chat history, retrieved documents, tool outputs — must fit inside this window.

Why this matters for agents: long conversations, large tool outputs, or big retrieved documents can silently overflow the context window, causing older information to be dropped or truncated. Agent frameworks manage this with summarization, truncation, or retrieval (RAG).

Temperature (and Sampling Parameters)

LLMs generate text by predicting a probability distribution over the next possible token, then sampling from it. **Temperature** controls how "random" that sampling is:

- **Low temperature (0–0.3):** more deterministic, focused, repeatable output. Good for factual tasks, code, tool-calling agents.
- **High temperature (0.7–1.0+):** more varied, creative, less predictable output. Good for brainstorming, creative writing.
- **top_p (nucleus sampling):** restricts sampling to the smallest set of tokens whose combined probability exceeds p . Often used alongside or instead of temperature.

For agents, low temperature is usually preferred — you want consistent, predictable tool calls and reasoning, not creative variation.

System / User / Assistant Roles

API calls to chat-based LLMs are structured as a list of role-tagged messages, not a single block of text:

Role	Purpose
------	---------

system	Sets the model's behavior, persona, and rules for the whole conversation. Set once, usually invisible to the user.
user	Represents input from the human (or, in an agent, sometimes a task description).
assistant	The model's own prior responses. Included in history so it has memory of what it already said.
tool / function	The result returned after the model calls a tool — fed back so the model can use it.

2. Prompt Engineering

Since agent reliability depends heavily on prompt quality, these three patterns are the core toolkit.

Zero-Shot Prompting

You give the model a task with **no examples** — just an instruction. Works well for simple, common tasks the model has already seen a lot of during training (e.g. "Summarize this email in two sentences").

Few-Shot Prompting

You include a **handful of example input/output pairs** in the prompt before asking the real question. This steers the model toward a specific format, tone, or reasoning style without any fine-tuning. Useful when you need a consistent output structure (e.g. JSON with specific fields, a particular writing style).

Chain-of-Thought (CoT) Prompting

You explicitly ask the model to **reason step by step** before giving a final answer (e.g. "Think through this step by step, then give your answer"). This improves performance on tasks involving logic, math, or multi-step decisions — which is exactly what agents do when deciding which tool to call and why.

In agent frameworks, this shows up as the **ReAct pattern** (Reason + Act): the model alternates between reasoning ("I need to check the weather first") and acting (calling a tool), observing the result, and reasoning again before the next step.

Other Useful Techniques

- **Role prompting:** "You are an expert Python debugger..." — framing improves task-appropriate responses.
- **Structured output prompting:** explicitly requesting JSON/XML output with a defined schema, critical for agents that parse LLM output programmatically.
- **Self-consistency:** running the same prompt multiple times and taking a majority-vote answer, for higher-stakes reasoning tasks.

3. Tool Calling / Function Calling

This is the single most important concept for building agents. Without it, an LLM can only talk — with it, an LLM can act.

What It Is

Tool calling lets you describe a set of functions (name, description, parameters/schema) to the model. When the model decides a function would help answer the user's request, instead of replying in plain text, it returns a structured request to call that function with specific arguments. Your code then actually runs the function and sends the result back to the model.

The Flow (step by step)

- 1 You send the model a prompt plus a list of available tools (name + description + parameter schema).
- 2 The model decides: reply directly in text, OR request a tool call with specific arguments.
- 3 If it requests a tool call, your application code (not the model) actually executes that function — e.g. hitting a weather API, querying a database, running a calculation.
- 4 The function's result is sent back to the model as a new message in the conversation.
- 5 The model reads the result and either calls another tool, or produces a final answer for the user.

Why It Matters for Agents

An "agent" is essentially this loop — reason, call a tool, observe the result, reason again — repeated until the task is done. Every agent framework (OpenAI Agents SDK, LangGraph, CrewAI, AutoGen) is built around this same core mechanism, just with different abstractions for managing the loop, memory, and multi-agent coordination.

Key insight: the LLM never executes code itself. It only decides *what* to call and *with what arguments* — your application is responsible for actually running it. This separation is what makes tool calling safe(r) and controllable.

MCP (Model Context Protocol)

Since your course covers MCP: it's a standardized protocol for exposing tools/data sources to LLMs in a consistent way, so you don't have to write custom tool-calling glue for every app/model combination — the same MCP "server" can plug into many different agents.